◆ FULL-STACK DEVELOPER BUILD GUIDE

EngineerTutorial Platform

Complete build reference — requirements, architecture, implementation sequence, and post-MVP roadmap.

Next.js 14 App Router | MongoDB + Mongoose | NextAuth.js JWT | Razorpay | Monaco Editor | Judge0 API | Tailwind CSS | TanStack Query | Vercel + Atlas

DOCUMENT VERSION	DATE	STATUS
1.0	**May 2026**	**MVP Build**

1 Platform Overview

What we're building, who it's for, and how it's different

What We're Building

A GFG/HelloInterview-style CS learning platform focused on System Design, Apache Kafka, LLD/HLD, AWS, and GenAI — with a built-in code editor, AI chatbot, and a subscription model. The admin panel is a fully customizable CMS (no code deploys needed to update content or navigation).

> **Reference repo:** `github.com/Deepak12214/Engineer-Tutorial` (React + Vite SPA, 60-70% landing/reading UI done). We're migrating/rebuilding as **Next.js 14 App Router + MongoDB**.

Key Differentiators

FEATURE	DETAILS
Block-Based CMS	All content stored as `blocks[]` JSON array (same format as reference repo). Admin edits without touching code.
Built-in Code Editor	Monaco Editor + Judge0 API. 8 languages, sandboxed execution, "Open in Playground" from any code block.
AI Chatbot Sidebar	Floating panel. Context-aware (injects current topic into system prompt). Unlimited & free for all users. Proxied to your separate AI service.
Server-Side Content Gating	API slices `blocks[]` server-side. Client blur overlay is UX only — not security.
Blogs Always Free	`isPremium` flag exists only on Topics. All blogs are always publicly readable.
Razorpay Payments	Indian gateway — UPI, cards, netbanking. HMAC-SHA256 signature verification server-side.
DB-Driven Navigation	Header, footer, announcement bar all stored in MongoDB `SiteConfig`. Change without deploy.

User Roles

ROLE	CONTENT ACCESS	PANEL ACCESS
guest	Free topics + all blogs (no login)	—
free_user	Free topics + preview of premium + all blogs	Dashboard only
premium_user	All topics + all blogs	Dashboard + billing

author	All	Content editor (no user mgmt)
admin	All	Full admin panel

Content Hierarchy

```
Platform
└── Course          (e.g., "System Design")       → /learn/system-design
    └── Section     (e.g., "Fundamentals")
        └── Topic   (e.g., "Load Balancing")      → /learn/system-design/fundamentals/load-balancing
Blog                                               → /blogs/[slug]   ← always free
```

2 Build Sequence

Build in this exact order — each step depends on the previous

Estimated time: 10–14 weeks for one developer. Steps 1–10 = MVP (shippable). Steps 11–15 = full feature set.

1 Project Setup + MongoDB Connection

`next.config.ts`, `tailwind.config.ts`, `globals.css` (CSS vars), `lib/mongodb.ts` singleton, Mongoose models (User, Course, Section, Topic, Blog, SiteConfig, Progress).

Deliverable: `npm run dev` connects to Atlas, models compile.

2 Authentication — NextAuth.js

CredentialsProvider (bcrypt), GoogleProvider, GithubProvider. JWT strategy. Role + subscriptionStatus in token. `middleware.ts` protects `/dashboard/*` and `/admin/*`.

Deliverable: Login/Register pages work. `/admin` redirects to login if unauthenticated.

3 Content APIs + Block Renderer

GET courses, sections, topics (premium-gated). `ContentRenderer` component that renders all 14 block types. Migrate existing JSON data to MongoDB via `scripts/migrate.mjs`.

Deliverable: Topic reading page displays real content from DB.

4 Landing + Public Pages

Homepage (hero, course cards, AI feature, FAQ), Blog listing, Blog detail, /learn course overview, /pricing page. SEO: `generateMetadata()` on each route.

Depends on: Step 3 (real content from DB)

5 Content Gating (Server-Side)

API checks JWT `subscriptionStatus + currentPeriodEnd`. Returns `blocks.slice(0, previewBlockCount)` for free users. `ContentGate` blur overlay on client. Login wall modal for guests.

Depends on: Steps 2 + 3 | Critical security step

6 User Dashboard + Progress Tracking

"Mark as Complete" API, progress bar per course, resume learning, bookmarks (CRUD). Dashboard page with enrolled courses, stats, recent blogs.

Depends on: Step 2 (auth)

7 Razorpay Subscription Flow

Create Order (server) → Razorpay checkout modal (client) → HMAC-SHA256 verify (server) → update User.subscriptionStatus = 'pro'. Webhook handler for async events. Upgrade modal component.

Depends on: Step 5 (gating must exist first)

8 Code Editor (Monaco + Judge0)

`@monaco-editor/react` wrapper. Judge0 proxy API. In-memory rate limiter (10 runs/min/IP). /playground page. "Open in Playground" button on code blocks. stdin field, output panel (stdout/stderr/time/memory).

Deliverable: Users can write and run code in 8 languages.

---

9 AI Chatbot Integration

Floating panel (open/closed/minimized). Context-aware system prompt. `/api/ai/chat` streaming proxy → your AI service. "Ask AI about this code" button on code blocks. No usage limits.

Depends on: Step 3 (topic context)

---

10 Admin Block Editor CMS

`@dnd-kit/core` drag-and-drop. Block palette (14 types). Inline editing. Live preview panel (reuses `ContentRenderer`). Metadata sidebar. Auto-save draft every 30s. On publish: `revalidatePath()` for on-demand ISR.

Most complex component — plan 2 weeks minimum

---

11 Admin Navigation Builder

Edit header links, dropdowns, CTA button, announcement bar (text/color/link/toggle), footer description. Saves to `SiteConfig` MongoDB doc. `revalidatePath('/', 'layout')` on save.

---

12 Admin SEO Manager

Per-content SEO fields editor (meta title, description, OG image). Global SEO settings (title template, default OG, Twitter handle). SEO completeness scores. "Regenerate Sitemap" button.

---

13 Admin User Management

User table (search/filter/paginate). Grant/revoke Pro. Change role. Ban user. View activity. Export CSV. Subscription list with Razorpay refund API integration.

---

14 SEO — Metadata, OG Images, Sitemap

`generateMetadata()` on all dynamic routes. `@vercel/og` for dynamic OG images. `app/sitemap.ts` queries MongoDB for all published content. JSON-LD structured data (Article, Course, BreadcrumbList, FAQPage).

---

15 Image Upload (Dual)

Option A: Upload to Cloudinary → get URL → insert in image block. **Option B:** Upload to `/public/uploads/` via backend → get path → insert in image block. Both available in admin panel image dialog.

---

16 Global Search (Ctrl+K)

MongoDB text index on Title + Tags. Search modal with grouped results (Topics / Blogs / Courses). Lock icon on premium results. Recent searches in localStorage.

---

17 Content Migration Script

`scripts/migrate.mjs` — reads JSON files from reference repo, maps to Mongoose schemas, seeds MongoDB. One-time run. Block format is **identical** — direct copy.

---

18 **Deploy: Vercel + MongoDB Atlas**

Import repo to Vercel. Add all env vars. Enable Atlas IP whitelist (`0.0.0.0/0`). Configure Razorpay webhook URL. Switch to Razorpay Live keys. Run Lighthouse audit. Ship.

3 Functional Requirements

Condensed — every critical behaviour in one place

3.1 Authentication & User Management

REQUIREMENT	DETAIL
Registration	Email + password (bcrypt hash). Google OAuth. GitHub OAuth. Email verification on signup.
Login / Session	NextAuth.js JWT. 30-day expiry. "Remember me" option. Password reset via OTP email.
Profile	Display name, bio, avatar, GitHub URL, LinkedIn URL, theme preference.
Progress	Per-course completion %, "resume learning" (last visited topic), sidebar checkmarks, daily streak.
Bookmarks	Save/remove any topic or blog. Listed in dashboard bookmarks tab.

3.2 Content — Block Format

All content (topics AND blogs) is an ordered `blocks[]` array. Each block: `{ type, content }`.

BLOCK TYPE	DESCRIPTION	BLOCK TYPE	DESCRIPTION
`heading`	H1 section title	`callout`	Info/Warning/Tip box
`smallHeading`	H2 subheading	`table`	Data table with headers
`text`	Paragraph + inline markdown	`image`	Image with alt + caption
`list`	Bullet list	`video`	YouTube embed or hosted
`orderedList`	Numbered list	`quiz`	MCQ with answer reveal
`code`	Syntax-highlighted code block	`badge`	Colored tag chips
`line`	Horizontal divider	`gap`	Vertical spacing

3.3 Content Gating Rules

> **Rule:** `isPremium` flag exists ONLY on Topic documents. Blogs have no such field — all blogs are always free for everyone including guests.

VISITOR	FREE TOPIC	PREMIUM TOPIC	ANY BLOG
Guest (not logged in)	✓ Full content	Preview + login wall	✓ Full content

Free user	✓ Full content	Preview + subscribe modal	✓ Full content
Premium user	✓ Full content	✓ Full content	✓ Full content

Preview = `blocks.slice(0, previewBlockCount)` returned by API. Default `previewBlockCount = 3`. Client blur overlay is UX only. Security is server-side.

3.4 Code Editor

FEATURE	IMPLEMENTATION
Editor engine	Monaco Editor (`@monaco-editor/react`) — same as VS Code
Languages	JavaScript, Python, Java, C++, Go, SQL, Rust, Bash
Execution	Judge0 API (sandboxed). 5s timeout, 256MB memory. Shows stdout/stderr/time/memory.
Rate limiting	Node.js in-memory Map — 10 runs/minute per IP. Resets on server restart.
Pages	Standalone `/playground` + inline runnable blocks in topics. "Open in Playground" button.
Theme	Auto dark/light based on site theme toggle.

3.5 AI Chatbot

FEATURE	DETAIL
Location	Floating panel, bottom-right. Open / minimized / closed states.
Cost to users	**Free and unlimited for all users** — no message caps, no login required.
Context injection	On topic pages: topic title + course name auto-injected into system prompt.
Code button	"Ask AI about this code" on code blocks pre-fills chat with the code snippet.
Streaming	API route proxies to your external AI service. Response streamed token-by-token (ReadableStream).
Shortcut	Ctrl+/ (Cmd+/) toggles panel. Persists chat history for the session (client state only).

3.6 Admin Panel Modules

MODULE	KEY CAPABILITIES
Dashboard	Stats (users, MRR, DAU, content count), revenue chart, recent activity feed
Block Editor CMS	Create/edit Topics & Blogs. Drag-and-drop blocks. Live preview. Auto-save draft (30s). On publish: `revalidatePath()`.
Image Upload	Two options: (1) Cloudinary upload → URL, (2) Static upload to `/public/uploads/` → path. Both in editor.

Navigation Builder	Header links (add/remove/reorder), dropdown sub-menus, CTA button, announcement bar (text/link/color/toggle). Saved to DB.
SEO Manager	Per-page meta title/description/OG image. Global SEO template. Completeness score per content item. Sitemap regeneration.
User Management	Search/filter/paginate users. Grant/revoke Pro. Change role. Ban. Export CSV.
Subscription Management	View active/failed subs. Issue manual Razorpay refund. Webhook logs.

3.7 Subscription & Payments (Razorpay)

PLAN	ACCESS	PRICE
Free	All free topics + all blogs	Free forever
Pro Monthly	All topics + all blogs	₹299/month
Pro Annual	All topics + all blogs	₹2,499/year

Payment Flow: User clicks "Upgrade" → Backend creates Razorpay Order → Client opens Razorpay modal → On success, client sends `{order_id, payment_id, signature}` → Backend verifies HMAC-SHA256 → Updates `User.subscriptionStatus = 'pro'` → User gets instant access.

No trial period. Free content is accessible to all. Premium access requires payment only.

4 Technical Architecture

Project structure, database schemas, API routes

4.1 Project Folder Structure

```
platform/
├── app/
│   ├── (public)/              # Landing, blogs, learn, pricing, playground
│   ├── (auth)/                # login, register, reset-password
│   ├── (dashboard)/           # /dashboard, /dashboard/bookmarks, /dashboard/billing
│   ├── (admin)/admin/         # /admin, /admin/content, /admin/navigation, /admin/seo, /admin/users
│   ├── api/
│   │   ├── auth/[...nextauth]/route.ts
│   │   ├── topics/[topicId]/route.ts       ← premium gating here
│   │   ├── blogs/, courses/, search/
│   │   ├── user/  (me, progress, bookmarks)
│   │   ├── ai/chat/route.ts                ← streaming proxy
│   │   ├── code/execute/route.ts           ← Judge0 proxy
│   │   ├── razorpay/ (create-order, verify-payment, webhook)
│   │   ├── upload/ (cloudinary, static)
│   │   └── admin/ (content, navigation, seo, users, stats)
│   ├── og/route.tsx                        ← dynamic OG images
│   ├── sitemap.ts                          ← auto-generated from DB
│   └── robots.ts
├── components/
│   ├── layout/     Header, Footer, CourseSidebar, Breadcrumbs
│   ├── content/    ContentRenderer, ContentGate, TableOfContents, TopicNav
│   ├── editor/     MonacoEditor, CodeRunner, BlockEditor
│   ├── ai/         AIChatWindow
│   └── subscription/ UpgradeModal, RazorpayButton
├── models/         User, Course, Section, Topic, Blog, SiteConfig, Progress
├── lib/            mongodb.ts, auth.ts, razorpay.ts, judge0.ts, rateLimiter.ts
├── context/        ThemeContext, ChatContext
└── middleware.ts   ← route protection
```

4.2 MongoDB Schemas (Key Fields)

Users		7 collections total
email	String	unique, indexed
role	enum	free_user | premium_user | author | admin | super_admin
subscriptionStatus	enum	free | pro | cancelled
currentPeriodEnd	Date	when Pro access expires (checked on every API request)
razorpayPaymentId	String	last successful payment ID
streak, lastActiveDate	Number, Date	for daily streak counter on dashboard

Topics		Core content document
blocks	[BlockSchema]	ordered array — each: { type, content }. Same format as reference repo.
isPremium	Boolean	default false. Controls content gating.
previewBlockCount	Number	default 3. How many blocks free users see.
courseId, sectionId	ObjectId	refs to Course and Section collections
status	enum	draft | published | archived. Only 'published' served to users.
metaTitle, metaDescription, ogImage	String	used by generateMetadata()

Blogs		No isPremium field — always free
blocks	[BlockSchema]	identical format to Topics
slug, title, subtitle	String	slug is unique index
coverImage, tags, rating	String, [String], Number	

SiteConfig		Key-value store for admin-controlled config
key	String	'navigation' or 'global-seo'. One doc per key.
data	Mixed	flexible JSON. Header links, footer, announcement bar, SEO defaults.

Progress		
userId + courseId	ObjectId	compound unique index
completedTopics	[ObjectId]	array of completed Topic IDs
completionPercentage	Number	0–100, updated on every mark-complete

4.3 Key API Routes

GET	/api/courses	All published courses — public
GET	/api/topics/[id]	Full topic (gated) — checks JWT on every call
GET	/api/blogs/[slug]	Full blog — always public, no auth check
GET	/api/search?q=	MongoDB text index — grouped results
POST	/api/user/progress	Mark topic complete — requires session
POST	/api/ai/chat	Streaming AI proxy — open to all, no limits
POST	/api/code/execute	Judge0 proxy — rate limited 10/min/IP
POST	/api/razorpay/create-order	Create Razorpay order server-side
POST	/api/razorpay/verify-payment	HMAC-SHA256 verify → update subscription
POST	/api/razorpay/webhook	Async Razorpay events (payment.captured)
PUT	/api/admin/navigation	Update SiteConfig + revalidatePath layout
PUT	/api/admin/content/topics/[id]	Update blocks[] + revalidatePath on publish
POST	/api/upload/cloudinary	Multipart → Cloudinary → returns URL
POST	/api/upload/static	Multipart → /public/uploads/ → returns path

5 Key Implementation Patterns

Copy-paste-ready critical code for the 5 most important systems

5.1 NextAuth Configuration (`lib/auth.ts`)

```
export const { handlers, auth } = NextAuth({
  providers: [
    CredentialsProvider({
      async authorize(credentials) {
        await connectDB();
        const user = await User.findOne({ email: credentials.email });
        if (!user || !await bcrypt.compare(credentials.password, user.hashedPassword)) return null;
        return { id: user._id, email: user.email, role: user.role,
                 subscriptionStatus: user.subscriptionStatus,
                 currentPeriodEnd: user.currentPeriodEnd };
      }
    }),
    GoogleProvider({ clientId: process.env.GOOGLE_CLIENT_ID!, ... }),
    GithubProvider({ clientId: process.env.GITHUB_CLIENT_ID!, ... }),
  ],
  session: { strategy: 'jwt' },
  callbacks: {
    async jwt({ token, user }) {
      if (user) {
        token.role = user.role;
        token.subscriptionStatus = user.subscriptionStatus;
        token.currentPeriodEnd = user.currentPeriodEnd;
      }
      return token;
    },
    async session({ session, token }) {
      session.user.id = token.sub!;
      session.user.role = token.role;
      session.user.subscriptionStatus = token.subscriptionStatus;
      return session;
    },
  },
});
```

5.2 Server-Side Content Gating (`app/api/topics/[topicId]/route.ts`)

> **This is the security boundary.** The blur overlay on the client is just UX. Gating must happen here.

```
export async function GET(req: Request, { params }) {
  await connectDB();
  const session = await auth();
  const topic = await Topic.findById(params.topicId).lean();
```

```
  if (!topic || topic.status !== 'published')
    return Response.json({ error: 'Not found' }, { status: 404 });

  const isPremiumUser =
    session?.user?.subscriptionStatus === 'pro' &&
    new Date(session.user.currentPeriodEnd) > new Date();  // check expiry

  const isAdmin = ['admin','super_admin','author'].includes(session?.user?.role ?? '');

  if (topic.isPremium && !isPremiumUser && !isAdmin) {
    return Response.json({
      ...topic,
      blocks: topic.blocks.slice(0, topic.previewBlockCount),  // ← slice server-side
      isGated: true,
      totalBlocks: topic.blocks.length,
    });
  }
  return Response.json({ ...topic, isGated: false });
}
```

5.3 Razorpay Payment Flow

Step 1 — Create Order (server)

```
// POST /api/razorpay/create-order
const amount = planType === 'annual' ? 249900 : 29900;  // paise (₹2,499 or ₹299)
const order = await razorpay.orders.create({ amount, currency: 'INR', receipt: `r_${Date.now()}` });
return Response.json({ orderId: order.id, amount, currency: 'INR', key: process.env.RAZORPAY_KEY_ID
```

Step 2 — Client opens modal

```
const rzp = new Razorpay({ key, amount, currency, order_id: orderId,
  handler: async (response) => {
    await fetch('/api/razorpay/verify-payment', { method: 'POST',
      body: JSON.stringify(response), headers: { 'Content-Type': 'application/json' } });
    window.location.reload();  // refresh → now premium
  }
});
rzp.open();
```

Step 3 — Verify signature (server) — CRITICAL

```
// POST /api/razorpay/verify-payment
const { razorpay_order_id, razorpay_payment_id, razorpay_signature } = await req.json();
const body = razorpay_order_id + '|' + razorpay_payment_id;
const expected = crypto.createHmac('sha256', process.env.RAZORPAY_KEY_SECRET!)
  .update(body).digest('hex');

if (expected !== razorpay_signature)
  return Response.json({ error: 'Invalid signature' }, { status: 400 });

const end = new Date(); end.setMonth(end.getMonth() + 1);
await User.findByIdAndUpdate(session.user.id, {
```

```
  subscriptionStatus: 'pro', currentPeriodEnd: end, razorpayPaymentId: razorpay_payment_id
});
return Response.json({ success: true });
```

5.4 AI Streaming Proxy (`app/api/ai/chat/route.ts`)

```
export async function POST(req: Request) {
  const { messages, contextTopic, contextCode } = await req.json();

  const systemPrompt = contextTopic
    ? `You are a CS tutor on EngineerTutorial. User is reading: "${contextTopic}".`
    : `You are a CS tutor. Answer questions about system design, Kafka, AWS.`;

  const aiResponse = await fetch(process.env.AI_SERVICE_URL!, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${process.env.AI_SERVICE_API_KEY}`,
               'Content-Type': 'application/json' },
    body: JSON.stringify({
      messages: [{ role: 'system', content: systemPrompt }, ...messages],
      stream: true
    }),
  });

  return new Response(aiResponse.body, {   // forward stream directly to client
    headers: { 'Content-Type': 'text/plain; charset=utf-8' },
  });
}
```

5.5 In-Memory Rate Limiter (`lib/rateLimiter.ts`)

```
const store = new Map<string, { count: number; resetAt: number }>();

export function checkRateLimit(key: string, limit: number, windowMs: number): boolean {
  const now = Date.now();
  const entry = store.get(key);
  if (!entry || now > entry.resetAt) {
    store.set(key, { count: 1, resetAt: now + windowMs });
    return true;
  }
  if (entry.count >= limit) return false;  // blocked
  entry.count++;
  return true;
}

// Usage in code execute route:
// const ip = req.headers.get('x-forwarded-for') ?? 'unknown';
// if (!checkRateLimit(`code:${ip}`, 10, 60_000)) return 429;
```

5.6 On-Demand ISR — Revalidate on Publish

```
// In admin PUT /api/admin/content/topics/[id]/route.ts
import { revalidatePath } from 'next/cache';
```

```
// After saving to MongoDB:
if (body.status === 'published') {
  revalidatePath(`/learn/${courseSlug}/${sectionSlug}/${topicSlug}`);
  revalidatePath('/sitemap.xml');
}

// For navigation changes (saves to SiteConfig):
// revalidatePath('/', 'layout');  ← revalidates entire layout tree
```

5.7 Middleware — Route Protection (`middleware.ts`)

```
import { auth } from '@/lib/auth';
import { NextResponse } from 'next/server';

export default auth((req) => {
  const { pathname } = req.nextUrl;

  if (pathname.startsWith('/dashboard') && !req.auth)
    return NextResponse.redirect(new URL('/login', req.url));

  if (pathname.startsWith('/admin')) {
    if (!req.auth) return NextResponse.redirect(new URL('/login', req.url));
    const role = req.auth.user?.role;
    if (!['admin','super_admin','author'].includes(role ?? ''))
      return NextResponse.redirect(new URL('/', req.url));
  }
  return NextResponse.next();
});

export const config = { matcher: ['/dashboard/:path*', '/admin/:path*'] };
```

5.8 Dynamic SEO Metadata

```
// app/(public)/learn/[courseId]/[sectionId]/[topicId]/page.tsx
export async function generateMetadata({ params }): Promise<Metadata> {
  const topic = await Topic.findOne({ slug: params.topicId }).lean();
  return {
    title: topic?.metaTitle || topic?.title,
    description: topic?.metaDescription,
    openGraph: {
      title:  topic?.metaTitle || topic?.title,
      images: [{ url: topic?.ogImage || `/og?title=${encodeURIComponent(topic.title)}` }],
      type:   'article',
    },
    alternates: { canonical: `/learn/${params.courseId}/${params.sectionId}/${params.topicId}` },
  };
}
```

6 Environment Variables

Create `.env.local` in the project root — never commit this file

MONGODB

```
MONGODB_URI=mongodb+srv://<user>:
<pass>@cluster.mongodb.net/engineer-tutorial
```

NEXTAUTH

```
NEXTAUTH_SECRET=<openssl rand -base64 32>
NEXTAUTH_URL=http://localhost:3000
```

GOOGLE OAUTH

```
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
```

GITHUB OAUTH

```
GITHUB_CLIENT_ID=
GITHUB_CLIENT_SECRET=
```

RAZORPAY

```
RAZORPAY_KEY_ID=rzp_test_...
RAZORPAY_KEY_SECRET=
```

JUDGE0 (CODE EXECUTION)

```
JUDGE0_API_URL=https://judge0-
ce.p.rapidapi.com
JUDGE0_API_KEY=
```

YOUR AI SERVICE

```
AI_SERVICE_URL=https://your-ai-
service.com/api/chat
AI_SERVICE_API_KEY=
```

CLOUDINARY

```
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
```

APP URL

```
NEXT_PUBLIC_APP_URL=https://yourdomain.com
```

NPM Packages to Install

```
npm install next-auth@beta mongoose bcryptjs
npm install razorpay
npm install @monaco-editor/react
npm install @dnd-kit/core @dnd-kit/sortable @dnd-kit/utilities
npm install cloudinary
npm install @tanstack/react-query

# Optional but recommended
npm install sharp          # faster Next.js image optimisation
npm install zod            # runtime validation for API request bodies
```

Deployment Checklist

STEP	ACTION
MongoDB Atlas	Create M0 cluster → create DB user → whitelist 0.0.0.0/0 → copy URI
Google OAuth	console.cloud.google.com → Credentials → OAuth → add redirect: `/api/auth/callback/google`
GitHub OAuth	github.com/settings/apps → New OAuth App → callback: `/api/auth/callback/github`
Razorpay	razorpay.com → Dashboard → Test Keys → copy KEY_ID and KEY_SECRET
Razorpay Webhook	Razorpay Dashboard → Webhooks → Add `https://yourdomain.com/api/razorpay/webhook` → enable `payment.captured`
Vercel Deploy	Import repo → paste all env vars → deploy. Every push to `main` = production deploy.
Go Live	Switch from `rzp_test_` keys to `rzp_live_` keys in Vercel env vars.

7 Post-MVP Roadmap

Features to add after the platform is live — phased by value and complexity

Ship rule: Launch with Steps 1–10 (auth + content + gating + payments + AI + code editor). Add Phase 2+ once you have real users giving feedback.

PHASE 2 · MONTHS 4–6

Community & Social

- Per-topic comment threads (upvote/downvote, threading)
- Private user notes + text highlights per topic
- Public profile pages (`/u/[username]`)
- In-app + email notification system (streak reminders, new content, sub expiry)

PHASE 3 · MONTHS 7–9

Gamification & Certificates

- XP system (topic +10, login +5, streak bonus)
- 5 levels: Novice → Learner → Practitioner → Expert → Master
- Badges (course completion, streaks, Pro member)
- Weekly leaderboard by XP
- Problem of the Day (POTD) with +25 XP
- PDF certificate on 100% course completion + LinkedIn share link

PHASE 4 · MONTHS 10–14

Video & Live Content

- Custom video player (chapters, playback speed, transcripts)
- Cloudinary / Mux video hosting for uploaded content
- Live coding sessions via Daily.co embed
- Recordings auto-published as premium content
- Interactive roadmap pages (like roadmap.sh) using ReactFlow

PHASE 5 · MONTHS 12–18

AI Expansion

- AI Mock Interviews (System Design, LLD, Behavioral) with scoring rubric
- AI Code Review — inline annotations, complexity analysis, diff view
- Skills Assessment Quiz → personalized learning path
- AI Resume Review (PDF upload → gap analysis → course suggestions)
- AI-generated practice problems per topic

PHASE 6 · MONTHS 18–24

Monetization Expansion

- Corporate team accounts (per-seat pricing, team progress reports)
- 1:1 Mentorship Marketplace (platform takes 20% cut via Razorpay)
- Job board with "Verified Skills" badges for course completions
- Company-specific interview prep packs (Google, Amazon, Meta)
- Content Licensing API + LTI for Canvas/Moodle

ONGOING · AS TRAFFIC GROWS

Technical Improvements

- **Search:** Replace MongoDB text index with Algolia or Meilisearch
- **Analytics:** PostHog — funnels, session recording, feature flags for A/B testing pricing
- **i18n:** next-intl for Hindi/Tamil/Telugu (huge Indian market)
- **PWA:** Offline reading cache, home screen install, background sync
- **Mobile:** React Native + Expo when PWA isn't enough
- **A/B Test pricing:** ₹199 vs ₹299 — PostHog feature flags

Decision Triggers — When to Add Each Phase

PHASE	START WHEN...
Phase 2 (Community)	DAU > 200 and users ask for comments/notes in feedback
Phase 3 (Gamification)	Retention < 40% week-2. Gamification directly targets this.
Phase 4 (Video)	Monthly revenue > ₹2L AND users ask for video content
Phase 5 (AI Mock)	Your AI service handles complex multi-turn conversations reliably
Phase 6 (Corporate)	First B2B inbound inquiry. Don't build it speculatively.
Algolia Search	Search latency > 800ms or relevance complaints from users
Mobile App	Mobile traffic > 50% AND PWA engagement scores are low